

Déploiement dissocié : recette & cliente

`staging` → serveur de dev (Contabo / horus-lab) · `main` → serveur cliente (DMZ). Le code, partie par partie. Màj 30/07/2026.

1. Vue d'ensemble

Un seul code, **deux environnements**, pilotés par la branche Git. Chaque branche produit sa « saveur » d'images Docker et vise son serveur.

`staging` → images `:staging` (domaines horus-lab) → **Contabo recette** ✓ **actif**
`main` → images `:main` (domaines cliente) → **Serveur cliente (DMZ)** ⌚ **à configurer**

Le concept-clé : une **image Docker** = le **code figé** + les **domaines figés dedans** (au build). On génère donc deux saveurs : `:staging` (domaines horus-lab) et `:main` (domaines cliente). Chaque serveur tire SA saveur — on ne mélange jamais. C'est pourquoi la recette doit tirer `:staging` (jamais `:latest`, qui portera les domaines cliente).

	Recette (horus-lab)	Cliente (DMZ)
Branche	<code>staging</code>	<code>main</code>
Images	<code>:staging</code>	<code>:main</code>
Proxy / TLS	nginx mutualisé (le nôtre)	leur Traefik (on s'y branche)
Base / Stockage	les nôtres (dans notre compose)	les leurs (base+bucket créés chez eux)
Conteneurs lancés	tout (db, backend, front...)	5 apps : backend, qcluster, site, portal, admin
Mécanisme	job inline <code>deploy-staging</code> de <code>ci.yml</code>	<code>deploy.yml</code> (guard <code>CLIENT_*</code>)

2. Le code partie par partie

2.1 — `ci.yml` : figer les bons domaines selon la branche

Au build du front, on injecte les domaines dans les images. C'est ici que se décide « saveur horus-lab » vs « saveur cliente ».

```
# .github/workflows/ci.yml - étape "Resolve build-time domains (per branch)"
if [ "${{ github.ref_name }}" = "main" ]; then
  SITE="${{ vars.CLIENT_SITE_DOMAIN || 'gathe-finance.horus-lab.com' }}"
  PORTAL="${{ vars.CLIENT_PORTAL_DOMAIN || 'portail.gathe-finance.horus-lab.com' }}"
else
  SITE="${{ vars.STAGING_SITE_DOMAIN || 'gathe-finance.horus-lab.com' }}"
  PORTAL="${{ vars.STAGING_PORTAL_DOMAIN || 'portail.gathe-finance.horus-lab.com' }}"
fi
```

→ `main` prend les variables `CLIENT_*` (repli horus-lab tant qu'elles sont vides). `staging` prend `STAGING_*`. Les images sont ensuite taguées `:main` / `:latest` (sur main) ou `:staging` (sur staging) via `docker/metadata-action`.

2.2 — `ci.yml` : le job inline qui déploie la RECETTE

Déjà présent dans le pipeline. Se déclenche uniquement sur un **push sur `staging`**.

```
# .github/workflows/ci.yml - job "deploy-staging"
deploy-staging:
  name: deploy staging (notre serveur)
  needs: [build-backend, build-frontend]
  if: github.ref == 'refs/heads/staging' && github.event_name == 'push'
  environment: { name: staging, url: ${vars.SITE_URL} }
  steps:
    # Guard : s'ignore si VPS_HOST vide
    # SSH vers Contabo, puis :
    NEW_TAG="staging"
    COMPOSE="docker compose -f docker-compose.prod.yml -f docker-compose.nginx-external.yml --env-file .env.p
    sed -i "s/^GATHE_IMAGE_TAG=.*GATHE_IMAGE_TAG=$NEW_TAG/" .env.prod # bascule le serveur sur :staging
    $COMPOSE pull && $COMPOSE up -d --force-recreate
    # healthcheck backend + rollback auto + docker restart backend-nginx-1
```

→ Les secrets `VPS_HOST/USER/SSH_PRIVATE_KEY` sont **repo-level** (déjà posés). C'est ce job qui a mis la nouvelle vitrine en ligne sur horus-lab. **Ne PAS créer de workflow séparé** (doubleton = double déploiement).

2.3 — `deploy.yml` : déploie la CLIENTE (avec garde-fou)

Se déclenche sur CI verte de `main`. Le **guard** le rend inoffensif tant que la cliente n'est pas configurée.

```
# .github/workflows/deploy.yml - garde-fou
- name: Guard - config client présente ?
  run: |
    if [ -z "${secrets.CLIENT_VPS_HOST}" ]; then
      echo "::warning::Config client absente - deploy ignoré."
      echo "skip=true" >> "$GITHUB_OUTPUT" # ← rien ne part
    else
      echo "skip=false" >> "$GITHUB_OUTPUT"
    fi
```

```
# .github/workflows/deploy.yml - déploiement SSH (si guard OK)
COMPOSE="docker compose -f docker-compose.prod.yml -f docker-compose.client.yml --env-file .env.prod"
APP_SERVICES="backend qcluster site portal admin" # ← seulement nos 5 apps
sed -i "s/^GATHE_IMAGE_TAG=.*GATHE_IMAGE_TAG=$NEW_TAG/" .env.prod # NEW_TAG = main
$COMPOSE pull $APP_SERVICES
$COMPOSE up -d --no-deps $APP_SERVICES # --no-deps = on NE lance PAS db/traefik
# healthcheck backend + rollback auto au tag précédent si échec
```

→ `--no-deps` + la liste explicite `APP_SERVICES` = on ne démarre que nos conteneurs applicatifs, jamais leur Postgres/Traefik.

2.4 — `docker-compose.client.yml` : le rattachement à la DMZ

C'est le fichier qui « débranche » notre base et raccorde nos conteneurs aux services de la cliente.

```
# infra/docker-compose.client.yml (combiné avec docker-compose.prod.yml)
networks:
  edge: { external: true, name: ${EDGE_NETWORK:-edge} } # réseau de LEUR Traefik
  data: { external: true, name: ${DATA_NETWORK:-data} } # réseau de LEURS bases

services:
  backend:
    networks: [internal, edge, data] # edge=ingress · data=Postgres/MinIO
    environment:
      DATABASE_URL: ${DATABASE_URL} # ← vers LEUR Postgres (base gathe_prod)
    depends_on: !reset [] # on coupe la dépendance à notre `db`
    labels:
      - "traefik.docker.network=${EDGE_NETWORK:-edge}" # dit à Traefik par où router

  qcluster: { networks: [internal, data], environment: { DATABASE_URL: ${DATABASE_URL} }, depends_on: !reset
  site: { networks: [internal, edge], labels: ["traefik.docker.network=${EDGE_NETWORK:-edge}"] }
  portal: { networks: [internal, edge], labels: ["traefik.docker.network=${EDGE_NETWORK:-edge}"] }
  admin: { networks: [internal, edge], labels: ["traefik.docker.network=${EDGE_NETWORK:-edge}"] }
```

→ Nos conteneurs rejoignent `edge` (leur Traefik les découvre via les labels `traefik.*` déjà dans `docker-compose.prod.yml` et émet le HTTPS) et `data` (accès Postgres/MinIO). `DATABASE_URL` écrase la connexion pour viser LEUR Postgres.

3. Config à saisir — GitHub

CONNU valeur fixe **RELEVÉ** via `inspect-dmz.sh` **CRÉÉ/GÉNÉRÉ**

3.1 — Variables (Settings → Secrets and variables → Actions → Variables)

Variable	Valeur	
<code>CLIENT_SITE_DOMAIN</code>	<code>app.gathe-finance.com</code>	CONNU
<code>CLIENT_PORTAL_DOMAIN</code>	<code>portail.gathe-finance.com</code>	CONNU
<code>CLIENT_API_DOMAIN</code>	<code>api.gathe-finance.com</code>	CONNU
<code>CLIENT_ADMIN_DOMAIN</code>	<code>admin.gathe-finance.com</code>	CONNU
<code>CLIENT_CMS_DOMAIN</code>	<code>cms.gathe-finance.com</code>	CONNU

3.2 — Secrets (onglet Secrets) — tu les saisis toi-même

Secret	Valeur
<code>CLIENT_VPS_HOST</code>	IP / hostname du serveur cliente (c'est lui qui « arme » le déploiement)
<code>CLIENT_VPS_USER</code>	utilisateur SSH (root ou deploy)
<code>CLIENT_VPS_SSH_PRIVATE_KEY</code>	clé privée SSH dédiée (la clé publique va dans <code>~/ .ssh/authorized_keys</code> du serveur)
<code>CLIENT_VPS_DEPLOY_PATH</code>	<code>= /home/gathe/gathe-finance</code> (<i>obligatoire ici : le repo n'est pas dans /opt</i>)

4. Config à saisir — serveur : `infra/.env.prod`

Copie de `infra/.env.prod.client.example`, à remplir SUR le serveur.

4.1 — Domaines **CONNU**

```

SITE_DOMAIN=app.gathe-finance.com
PORTAL_DOMAIN=portail.gathe-finance.com
ADMIN_DOMAIN=admin.gathe-finance.com
API_DOMAIN=api.gathe-finance.com
CMS_DOMAIN=cms.gathe-finance.com
DJANGO_ALLOWED_HOSTS=app.gathe-finance.com,portail.gathe-finance.com,admin.gathe-finance.com,api.gathe-finance.com
COOKIE_DOMAIN=.gathe-finance.com
PUBLIC_BASE_URL=https://api.gathe-finance.com

```

4.2 — Rattachement DMZ **RELEVÉ** (via `bash infra/scripts/inspect-dmz.sh`)

```

EDGE_NETWORK=..... # réseau Docker du Traefik
DATA_NETWORK=..... # réseau Docker des bases
TRAEFIK_CERTRESOLVER=..... # nom du resolver ACME (souvent letsencrypt)
TRAEFIK_ENTRYPOINT=websecure # nom de l'entrypoint 443 (à confirmer)

```

4.3 — Base de données **CRÉÉ** (dans LEUR Postgres)

```

# DATABASE_URL = postgres://<user>:<mdp>@<alias_postgres>:5432/<base>
DATABASE_URL=postgres://gathe:<mot_de_passe>@<alias_postgres>:5432/gathe_prod

```

4.4 — Stockage médias **CRÉÉ** (dans LEUR MinIO)

```

AWS_STORAGE_BUCKET_NAME=gathe-media
AWS_ACCESS_KEY_ID=.....
AWS_SECRET_ACCESS_KEY=.....
AWS_S3_ENDPOINT_URL=..... # ex. http://<alias_minio>:9000 (interne data)
AWS_S3_REGION_NAME=us-east-1
MEDIA_DOMAIN= # optionnel (ex. s3.gathe-finance.com)

```

4.5 — Secrets Django **GÉNÉRÉ**

```

DJANGO_SECRET_KEY=..... # python -c "import secrets;print(secrets.token_urlsafe(64))"
REVALIDATE_SECRET=..... # idem
DJANGO_SUPERUSER_USERNAME=admin
DJANGO_SUPERUSER_EMAIL=admin@gathe-finance.com
DJANGO_SUPERUSER_PASSWORD=.....

```

4.6 — Email Brevo **FOURNI** + Paiement + Tag

```

BREVO_API_KEY=.....
DEFAULT_FROM_EMAIL="Gathé Finance <noreply@gathe-finance.com>"
CONTACT_NOTIFICATION_EMAIL=contact@gathe-finance.com

DEFAULT_PAYMENT_PROVIDER=tara
TARA_API_KEY= # vide = mode mock (aucun paiement réel)
TARA_BUSINESS_ID=
TARA_WEBHOOK_SECRET=

GATHE_IMAGE_TAG=main # 🖼 la cliente tire les images :main

```

5. Actions serveur (hors `.env.prod`)

```
# ① Relevé DMZ (lecture seule)
bash /home/gathe/gathe-finance/infra/scripts/inspect-dmz.sh

# ② Créer NOTRE base dans LEUR Postgres
psql -U postgres -c "CREATE DATABASE gathe_prod;"
psql -U postgres -c "CREATE USER gathe WITH PASSWORD '*****';"
psql -U postgres -c "GRANT ALL PRIVILEGES ON DATABASE gathe_prod TO gathe;"

# ③ Créer NOTRE bucket + clés dans LEUR MinIO
mc mb local/gathe-media
mc admin user svacct add local <compte>      # -> access key + secret

# ④ Accès au registre d'images privé
docker login ghcr.io -u <user>                # PAT read:packages

# ⑤ DNS : app./api./cms./portail./admin.gathe-finance.com -> IP serveur (AVANT 1er start, pour l'ACME)

# ⑥ Post-déploiement (après 1re mise en ligne) : seeds + AppSettings
bash /home/gathe/gathe-finance/infra/scripts/post-deploy-seed.sh
```

6. Procédure d'ordre + vérification

1. Relevé DMZ (§5①) → remplir `.env.prod` §4.2.
2. Créer base + bucket (§5②③) → remplir §4.3 & §4.4.
3. Générer secrets Django (§4.5), poser Brevo (§4.6), `docker login` (§5④), DNS (§5⑤).
4. GitHub : Variables §3.1 + Secrets §3.2 (dont `CLIENT_VPS_HOST` = arme le déploiement).
5. **Merge sur `main`** → CI build `:main` → `deploy.yml` déploie la cliente.
6. Post-deploy seeds (§5⑥).

Vérification cliente

```
curl -sL -o /dev/null -w "%{http_code}\n" https://app.gathe-finance.com/
curl -sL -o /dev/null -w "%{http_code}\n" https://api.gathe-finance.com/healthz/
```

Attendu : `200`. Leur Traefik émet les certificats automatiquement dès que le DNS pointe et que nos conteneurs sont sur le réseau `edge` avec leurs labels.

Garde-fou : tant que `CLIENT_VPS_HOST` est vide, `deploy.yml` s'ignore → merger sur `main` reste 100 % sans risque avant la config.